

Capítulo 6 — GUÍA DE LECTURA

Capítulo 6 — GUÍA DE LECTURA.....	1
TypeScript: tipos sobre JavaScript y el contrato de la API	2
Los tipos, el borrado y el garante de RN-F08, explicados en criollo	2
Antes de empezar: cómo usar esta guía.....	2
6.1 — De qué se trata esta clase	3
Qué dice	3
En criollo.....	3
Por qué este capítulo es el sexto y no el primero	3
La conexión directa con el TPI	3
6.2 — Por qué existe TypeScript: el problema que vino a resolver	4
Qué dice	4
Los tres intentos anteriores, y por qué cada uno falló distinto	4
Las cuatro decisiones que explican todo el capítulo	4
6.3 — Compilar y borrar: qué queda cuando el navegador ejecuta	5
Qué dice	5
En criollo.....	5
6.4 — Anotar, inferir, y los tres tipos especiales.....	7
6.4.1 — Anotar lo que hace falta, y dejar que el resto se infiera.....	7
6.4.2 — <code>any</code> , <code>unknown</code> y <code>never</code> : tres formas de decir «no sé»	7
6.5 — La forma de un objeto: interfaces y alias.....	8
Acá se ve el tipado estructural de la sección 6.2, en tres líneas	8
6.6 — Uniones, estrechamiento y guardas.....	9
Qué dice	9
En criollo.....	10
6.7 — Genéricos, con moderación	11
6.8 — El modo estricto: qué prende cada bandera	12
6.9 — Del contrato del TPI al tipo	13
6.9.1 — Traducir un esquema de la sección 7	13
6.9.2 — El dinero, y por qué el tipo es el garante de RN-F08	14
6.9.3 — Que un identificador no se pueda pasar por otro	14
6.9.4 — Cómo queda la capa <code>api/</code>	15
6.10 — Dónde mienten los tipos.....	15
6.10.1 — El borde de la red.....	15
6.10.2 — Validar en el borde, una sola vez.....	16
6.11 — Herramientas de diagnóstico.....	17
6.12 — Seguridad y evolución.....	18
6.13 — Verificación: el checklist honesto.....	19

6.14 — Los once errores frecuentes	19
6.15 — Las actividades, y qué busca cada una	20
1. Del esquema al tipo	20
2. El garante en acción	20
3. Exhaustividad obligatoria	20
4. Capa <code>api</code> / tipada	20
5. Identificadores distinguibles	20
6. Exploración: dónde mienten los tipos.....	20
7. Exploración: la solidez que no está.....	21
6.16 — Síntesis: las once frases	21
6.17 — Qué leer, y en qué orden	21
Si lees una sola cosa.....	21
Si lees tres	22
Las fuentes normativas (para consultar, no para leer de corrido)	22
Cierre: las seis cosas que hay que recordar	22

TypeScript: tipos sobre JavaScript y el contrato de la API

Los tipos, el borrado y el garante de RN-F08, explicados en criollo

Traducción didáctica del texto académico, sección por sección. Mismos conceptos, otro idioma.

Antes de empezar: cómo usar esta guía

Este documento no reemplaza al capítulo original: lo traduce. El académico está escrito en el idioma de los papers —denso, comprimido—; esta guía lo desarma y lo cuenta como se lo contarías a alguien en un café. La regla es una sola: **no se pierde ni un concepto**.

Cada sección tiene tres partes:

- **Qué dice** — la idea del original, en dos o tres oraciones.
- **En criollo** — la explicación larga, con la analogía que la hace pegar.
- **Para el pizarrón** — la frase que te tenés que llevar.

En las secciones operativas se va directo al grano.

LA IDEA MADRE DE TODO EL CAPÍTULO

Si te quedás con una sola frase, que sea esta:

TypeScript no te promete que si compila, funciona. Te promete que no te vas a olvidar de lugares.

No es humildad: está en sus objetivos de diseño, en la lista de cosas que **decidieron no hacer**. Renunció a ser demostrablemente correcto a cambio de describir el JavaScript que la gente realmente escribe.

De ahí las dos mitades del capítulo: las secciones 6.2 a 6.9 muestran lo que la herramienta sí te da, y la 6.10 **dónde los tipos mienten**.

6.1 — De qué se trata esta clase

Qué dice

Los cinco capítulos anteriores construyeron una aplicación posible: se pueden pedir datos, mostrarlos, reaccionar a eventos y recibir avisos del servidor. Todo eso funciona sin TypeScript, y la pregunta que abre este capítulo es **por qué el TPI lo exige igual**. Al finalizar, el alumno debe poder traducir un esquema de la sección 7 del TPI a tipos, escribir con ellos la capa de acceso a la API, y explicar por qué el compilador no lo protege de una respuesta que no coincida.

En criollo

La respuesta que te van a dar en cualquier lado —«para evitar errores»— es incompleta y por eso no convence: que JavaScript te deje sumar un número y un texto es feo, pero no es lo que te va a arruinar el TPI.

El problema es otro, más aburrido y mucho más caro: **a partir de cierto tamaño, nadie se acuerda de qué forma tiene cada objeto que circula por el programa**. ¿Pedidos trae `estado` o `estado_actual`? ¿`direccion` es un objeto o un identificador? ¿`total` puede venir nulo? En doscientas líneas eso se responde mirando; en un sistema con setenta endpoints y veintitrés entidades —el TPI— se responde mal, y el error aparece en ejecución, **tarde y lejos de su causa**.

Por qué este capítulo es el sexto y no el primero

Ya se anunció al cerrar el Capítulo 3: **para entender una solución hay que haber tenido el problema**. En el Capítulo 4 algo se rompió porque `querySelector` devolvió `null`; en el 5 confiaste de memoria en qué campos traía una respuesta; en el 3 viste que el tipo numérico no representa exactamente ciertos decimales. **Todos esos son errores que un compilador habría atajado antes de que existieran**. El que aprende TypeScript antes que JavaScript anota **para que el editor deje de subrayar**, sin saber qué previene.

Si no sabés esto...	...no vas a entender esto otro
Que los tipos se borran	Por qué <code>total: string</code> no impide que el servidor mande otra cosa, y por qué hay que validar en el borde
Que el sistema no es sólido a propósito	Por qué «compila» y «anda» son afirmaciones distintas, y por qué existe la sección 6.10
Que el <code>string</code> de un importe es el garante de RN-F08	Por qué corregir un <code>total: number</code> escrito por una IA es la diferencia entre facturar bien y acumular centavos de error

La conexión directa con el TPI

La **RN-F08** dice que el dinero viaja como cadena decimal, está fundada en el modelo numérico de la sección 3.5.4, y **declara quién la hace cumplir**:

Garante: revisión, y **el tipo TypeScript, que declara `string`**.

El mecanismo que este capítulo enseña **es lo que hace cumplir esa regla**: lo que impide que alguien escriba `total * cantidad` sin darse cuenta. Ya lo viste venir: en la sección 1.5 del Capítulo 1, `"total": "4750.00"` aparecía entre comillas y dijimos que el Capítulo 6 lo iba a desarrollar. Y la contracara conviene anticiparla: **TypeScript no promete que si compila, funciona**, y su documentación de diseño declara como objetivo *no perseguido* un sistema de tipos demostrablemente correcto.

PARA EL PIZARRÓN

Al terminar tenés que poder abrir **la sección 7 del TPI** y **traducirla a tipos**, justificando cada decisión. Y además explicar por qué, con todos esos tipos escritos, el compilador **no te protege** de que el servidor mande otra cosa. La primera mitad sin la segunda produce falsa confianza, peor que no tener tipos.

6.2 — Por qué existe TypeScript: el problema que vino a resolver

Qué dice

Para 2010, JavaScript ya no se usaba para validar formularios. Gmail, Google Maps y las aplicaciones que siguieron a la técnica del Capítulo 5 tenían decenas de miles de líneas, y aparecieron problemas que el lenguaje no había sido diseñado para enfrentar: no había forma de saber qué recibía una función sin leer su cuerpo, ni de encontrar todos los usos de una propiedad, ni de renombrar con seguridad. Microsoft anunció TypeScript el 1 de octubre de 2012, con Anders Hejlsberg —Turbo Pascal, Delphi, C#— al frente.

Los tres intentos anteriores, y por qué cada uno falló distinto

Hubo tres, y **por qué fallaron explica la forma que TypeScript tomó**.

El intento	Qué proponía	Por qué no alcanzó
ECMAScript 4 · hasta 2008	Tipado estático en el lenguaje mismo	Se abandonó en 2008 (Capítulo 3), y con él la idea de que la solución viniera del estándar
Closure Compiler · Google	Tipos en comentarios , verificados aparte	El editor no los usaba y nada obligaba a mantenerlos al día: un tipo que puede quedar desactualizado es una nota, no un tipo
Dart · Google, 2011	Reemplazar JavaScript por un lenguaje con tipos	La lección central: nadie iba a reescribir su aplicación entera para usarlo

Las cuatro decisiones que explican todo el capítulo

La apuesta de Hejlsberg partió del fracaso de Dart. **Todo lo raro que veas más adelante sale de alguna de estas cuatro**.

Decisión	Qué gana	Qué cuesta
1. Es un superconjunto de JavaScript: todo <code>.js</code> válido es un <code>.ts</code> válido	No hay migración: cambiás la extensión y agregás tipos donde convenga. Eso resolvió la adopción que hundió a Dart	Arrastra las rarezas del Capítulo 3: no las arregla, las describe
2. El tipado es estructural, no nominal: compatibles si tienen la misma forma	Describe el código real: los objetos de JavaScript no tienen clase en el sentido clásico (sección 3.8)	Dos tipos con significados distintos quedan intercambiables. Lo arregla la sección 6.9.3
3. Los tipos se borran: el compilador los quita y emite JavaScript común	No hay costo: ni biblioteca en ejecución, ni peso extra	No hay garantía: nada de lo declarado existe cuando el programa corre (6.10)

Decisión	Qué gana	Qué cuesta
4. El sistema no es sólido, a propósito , y así lo declaran los objetivos de diseño	Tipar patrones razonables que un sistema estricto prohibiría	Admite operaciones que fallan en ejecución. No es un error: es lo que decidieron

PARA ENTENDER: el corrector ortográfico, no el juez

La cuarta decisión es la que más te va a servir y la que casi nadie te explica. **TypeScript no te promete que si compila, anda.** Nunca lo prometió.

¿Por qué renunciaron a eso? Porque un sistema que no te deja mentir nunca **no puede describir el JavaScript que la gente realmente escribe**. Habrían tenido un lenguaje impecable que nadie podía usar sobre su código: el mismo intercambio de Dart, y ya sabían cómo terminaba.

¿Qué es, entonces? **El corrector ortográfico de un procesador de textos**. Te subraya las palabras mal escritas y en eso es buenísimo, pero **no te dice si lo que escribiste es verdad**. Y vuelve la pregunta del Capítulo 1: **¿qué resignó para funcionar?** La corrección demostrable. Dart no resignó nada y nadie lo adoptó.

6.3 — Compilar y borrar: qué queda cuando el navegador ejecuta

Qué dice

El navegador no ejecuta TypeScript. Un compilador transforma el código y emite JavaScript, y esa transformación tiene **dos partes independientes**: la **verificación de tipos**, que analiza las anotaciones y reporta inconsistencias, y la **emisión**, que las quita y produce JavaScript. Por defecto el compilador emite JavaScript **aunque haya encontrado errores de tipo**.

En criollo

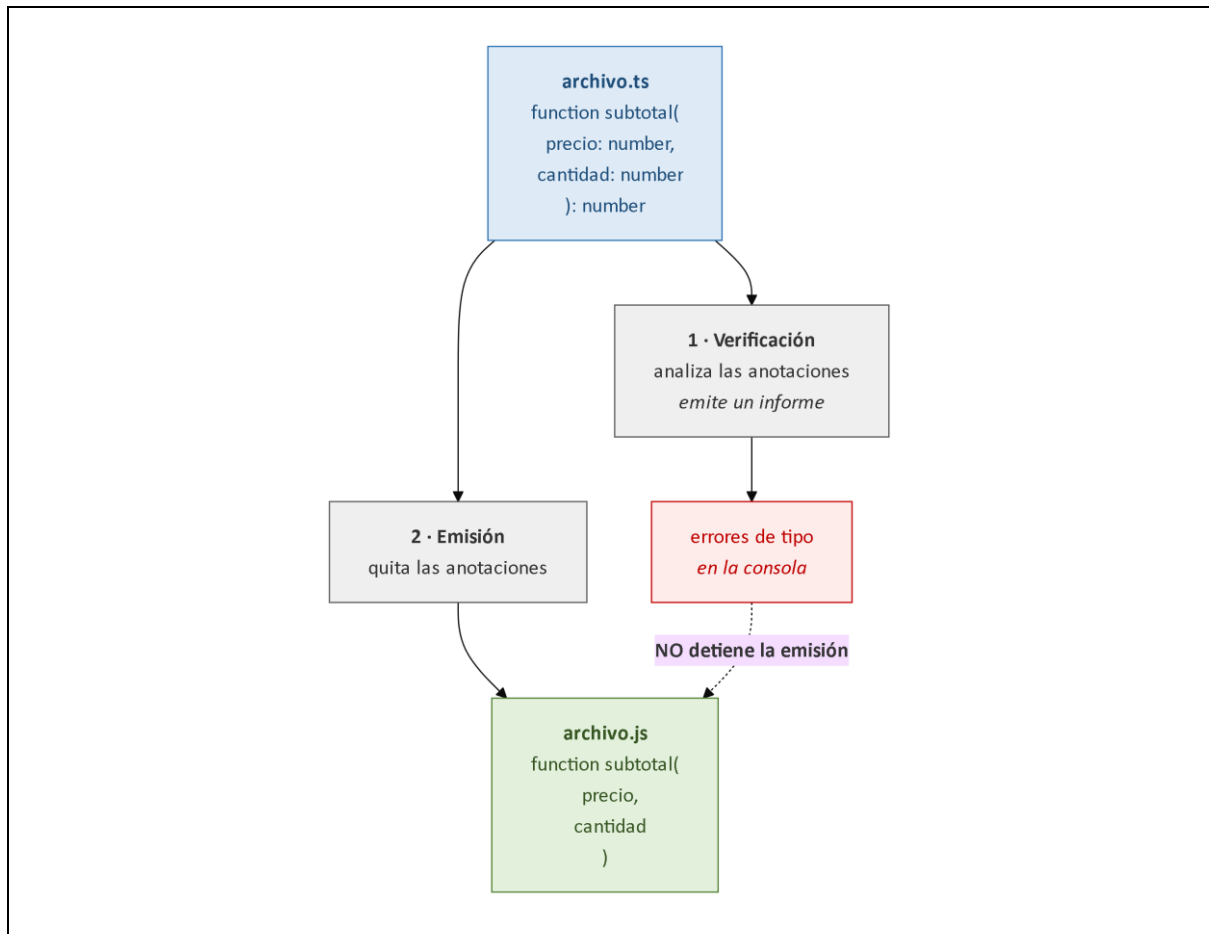
Eso sorprende a quien viene de un lenguaje compilado, donde un error de tipos frena todo. Acá **la verificación es un informe, no una traba**.

```
// Lo que se escribe
function calcularSubtotal(precio: number, cantidad: number): number {
  return precio * cantidad;
}

// Lo que se ejecuta
function calcularSubtotal(precio, cantidad) {
  return precio * cantidad;
}
```

Las anotaciones no están: ni un comentario, ni una marca, ni una función que verifique.

Desaparecieron.

Figura 6.1. Del `.ts` al `.js`: el borrado**PARA ENTENDER: las marcas de lápiz del carpintero**

Un carpintero que va a cortar una tabla **la marca antes con lápiz**: la línea de corte, dónde van los tornillos, cuál es la cara de arriba. Cuando el mueble está terminado, **lija y las marcas no están**.

Los tipos son eso: **te sirven mientras construí y no existen en el producto entregado**. De ahí las dos consecuencias juntas: **no pesan nada y no sostienen nada**.

La consecuencia del borrado	Qué significa cuando escribís código
No se puede preguntar por un tipo en ejecución	No existe <code>if (x es Pedido)</code> : en ejecución hay objetos , no tipos. Si lo necesitás, lo comprobás campo por campo (6.10.2)
Los tipos no cuestan nada	Lo que llega al navegador es idéntico a lo escrito a mano
Los tipos no validan nada	Declarar que algo es un <code>Pedido</code> no hace que lo sea

El Capítulo 7 estudia la herramienta que el TPI usa para compilar, y conviene adelantar un detalle: **esa herramienta no verifica los tipos**. Los borra y sigue de largo, para ser rápida. De ahí sale una situación que desconcierta: **el proyecto arranca perfecto con errores adentro**.

EXPERIMENTO — hacelo hoy, antes de seguir leyendo

1. Escribí un `.ts` con una función tipada y un error deliberado: pasarle un `string` donde va un `number`.
2. Compílalo con `tsc archivo.ts`.

3. Vas a ver el error en la consola. **Y ahora fijate que el .js se generó igual.**

4. Abrilo: las anotaciones no están, es JavaScript común.

5. Ejecutalo — **pero el error de tipo no aparece por ningún lado.**

Si tu proyecto arranca bien, **eso no significa que no tenga errores de tipo**. Hay que correr la verificación aparte.

6.4 — Anotar, inferir, y los tres tipos especiales

6.4.1 — Anotar lo que hace falta, y dejar que el resto se infiera

```
const nombre: string = "Milanesa";    // anotación explícita
const nombre = "Milanesa";           // inferido: string
```

La segunda es preferible cuando el tipo es evidente: anotar lo obvio agrega ruido y **una cosa más que puede quedar desactualizada**. La anotación vale en los **límites** —parámetros, retornos públicos, estructuras que cruzan módulos—: ahí no describe lo que ya se ve, **declara un contrato**.

```
let estado = "pendiente";             // string – se puede reasignar
const estado = "pendiente";           // "pendiente" – tipo literal
```

Con `let` infiere `string`; con `const`, **el valor exacto**, no la categoría. Ese comportamiento **es la base de las uniones literales** de la sección 6.9.1.

6.4.2 — `any`, `unknown` y `never`: tres formas de decir «no sé»

Tipo	Significa	Efecto
<code>any</code>	"No verifiques nada"	Desactiva el sistema de tipos para ese valor
<code>unknown</code>	"No sé qué es"	Obliga a estrechar antes de usarlo
<code>never</code>	"Esto no puede ocurrir"	El tipo vacío

La diferencia entre los dos primeros parece sutil y es opuesta:

```
function procesar(dato: any) {
  dato.loQueSea.queNoExiste();    // compila; explota en ejecución
}

function procesar(dato: unknown) {
  dato.loQueSea;                  // error de compilación: hay que verificar antes
  if (typeof dato === "object" && dato !== null && "id" in dato) {
    dato.id;                      // ahora sí
  }
}
```

Las dos dicen «no sé qué me están pasando»: **la primera se rinde, la segunda te obliga a averiguarlo**.

`any` es un agujero en el sistema de tipos y se propaga: todo lo derivado también lo es, así que **uno solo mal puesto desactiva la verificación de una rama entera**. `unknown` expresa la misma incertidumbre **sin renunciar a la verificación**: es un paquete cerrado que no podés usar hasta abrirlo. Por eso es **el tipo correcto para todo dato que entra desde afuera** (sección 6.10).

OJO ACÁ: ``any`` no es un tipo flexible, es un agujero — y se agranda solo

```
const config: any = leerConfiguracion();
const limite = config.paginacion.limite;    // any
const paginas = calcular(limite);           // any
```

```
const texto = paginas.toUpperCase(); // compila. Y paginas es un número.
```

Un solo any al principio desactivó la verificación de toda la cadena. Nadie escribió any en las tres líneas siguientes; **lo heredaron**. Y no te lo avisa nadie, porque técnicamente **no hay error**.

Por eso el hábito más útil que podés adoptar es este: **pasá el cursor por tus variables y fijate si alguna quedó en any sin que la pidieras**. Después buscá para atrás hasta el any original: **ahí está el problema, no donde explotó**.

Queda never, el más raro: aparece como retorno de una función que **nunca termina normalmente** y, lo más útil, para **verificar exhaustividad** (sección 6.6).

6.5 — La forma de un objeto: interfaces y alias

```
interface Producto {
  id: number;
  nombre: string;
  descripcion?: string; // opcional
  readonly creado_en: string; // sólo lectura
}

type Producto = {
  id: number;
  nombre: string;
};
```

El signo de pregunta declara que la propiedad **puede no venir**; readonly, que **una vez creado el objeto ese campo no se toca**.

La diferencia	Qué implica	Cuándo importa
interface admite fusión de declaraciones	Dos con el mismo nombre se combinan	Extender tipos de una biblioteca ajena; molesta si pasa sin querer
type puede ser cualquier cosa	Uniones, intersecciones, tipos calculados	Una unión literal como EstadoPedido tiene que ser type

Fuera de eso son intercambiables: **en el 95 % de los casos no importa cuál elijas, importa ser consistente**. Un archivo con las dos formas mezcladas al azar hace pensar que la diferencia significa algo, y el que lo lee pierde media hora buscando un criterio que no existía. Regla para el TPI: **interface para las respuestas de la API, type para los estados y todo lo demás**.

Acá se ve el tipado estructural de la sección 6.2, en tres líneas

```
interface Punto { x: number; y: number; }
interface Coordenada { x: number; y: number; }

const p: Punto = { x: 1, y: 2 };
const c: Coordenada = p; // válido: misma forma
```

En Java sería un error. **En TypeScript no, porque importa la forma y no el nombre**.

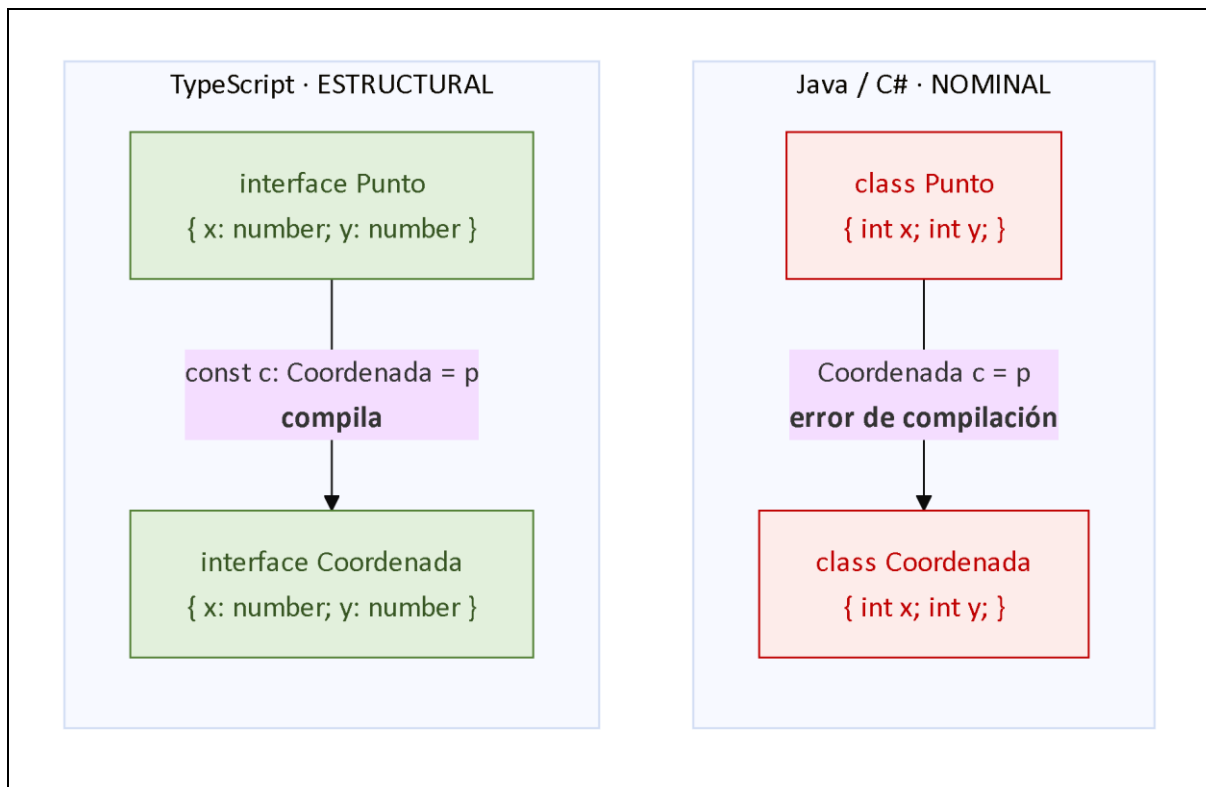


Figura 6.2. Tipado estructural frente a nominal

PARA ENTENDER: la llave y la cerradura

Un sistema nominal es un guardia con una lista. Te mira el carnet y si no estás anotado no entrás, aunque seas exactamente la persona que tenían que dejar pasar: verifica **la identidad declarada**.

Un sistema estructural es una cerradura. No le importa qué dice grabado en tu llave ni quién te la dio: **le importa la forma de los dientes**. Si encaja, abre.

TypeScript es la cerradura, y era la única forma de **describir el JavaScript real** (sección 3.8). El costo también se ve: **dos llaves iguales abren la misma puerta aunque una fuera del galpón y la otra de la caja fuerte**.

Hay una excepción que confunde la primera vez. Al asignar un **objeto literal** directamente, el compilador **rechaza las propiedades de más**:

```
const p: Punto = { x: 1, y: 2, z: 3 }; // error: 'z' no existe en Punto

const temp = { x: 1, y: 2, z: 3 };
const q: Punto = temp; // válido: ya no es un literal
```

Las dos líneas hacen lo mismo y una compila y la otra no. La razón es pragmática: una propiedad de más en un literal **casi siempre es un error de tipo**.

6.6 — Uniones, estrechamiento y guardas

Qué dice

Una **unión** expresa que un valor es una cosa u otra, y para usarlo hay que **estrechar**: convencer al compilador de cuál de las alternativas tiene en la mano. El caso más potente es la **unión discriminada**: formas que comparten una propiedad con valor literal distinto, y que permite verificar exhaustividad con `never`.

En criollo

```
type EstadoPedido = "pendiente" | "confirmado" | "en_preparacion" | "entregado" | "cancelado";
```

Esa es la **forma correcta de representar la máquina de estados de la sección 3.4 del TPI**. Con string a secas, **"entregdo" compila**; con la unión literal el compilador lo rechaza **y el editor autocompleta los cinco valores válidos**. Todo se apoya en la sección 6.4.1: los literales existen porque const infiere el valor exacto.

Guarda	Sirve para
<code>typeof x === "string"</code>	Primitivos
<code>x instanceof Error</code>	Clases
<code>"codigo" in x</code>	Presencia de una propiedad
<code>x !== null</code>	Descartar ausencia
<code>x.tipo === "exito"</code>	Uniones discriminadas

El «Adivina quién» del compilador

El estrechamiento es el juego del «Adivina quién»: un tablero lleno de caritas que van bajando con cada pregunta. Al principio tu valor **puede ser cualquiera de las alternativas**; cada guarda es una pregunta —«¿es una cadena?», «¿tiene codigo?»— y **cada respuesta baja las descartadas**.

Lo importante: **te sigue el razonamiento sólo dentro de un if o un switch que él pueda leer**. Si «sabés» cuál es porque lo averiguaste tres funciones más arriba, él no bajó ninguna carita.

Una **unión discriminada** es un conjunto de formas que comparten una propiedad —el *discriminante*— con valor literal distinto en cada una:

```
type Resultado =
  | { estado: "cargando" }
  | { estado: "exito"; pedidos: Pedido[] }
  | { estado: "error"; mensaje: string };

function render(r: Resultado) {
  switch (r.estado) {
    case "cargando": return mostrarCargando();
    case "exito":    return mostrarLista(r.pedidos); // acá sí existe .pedidos
    case "error":   return mostrarError(r.mensaje); // acá sí existe .mensaje
  }
}
```

Dentro de cada rama **el compilador sabe qué propiedades existen**, y eso hace imposible leer `r.pedidos` en la rama de error. Es la forma correcta de modelar cualquier cosa que esté **en uno de varios estados posibles**, que en una interfaz es casi todo.

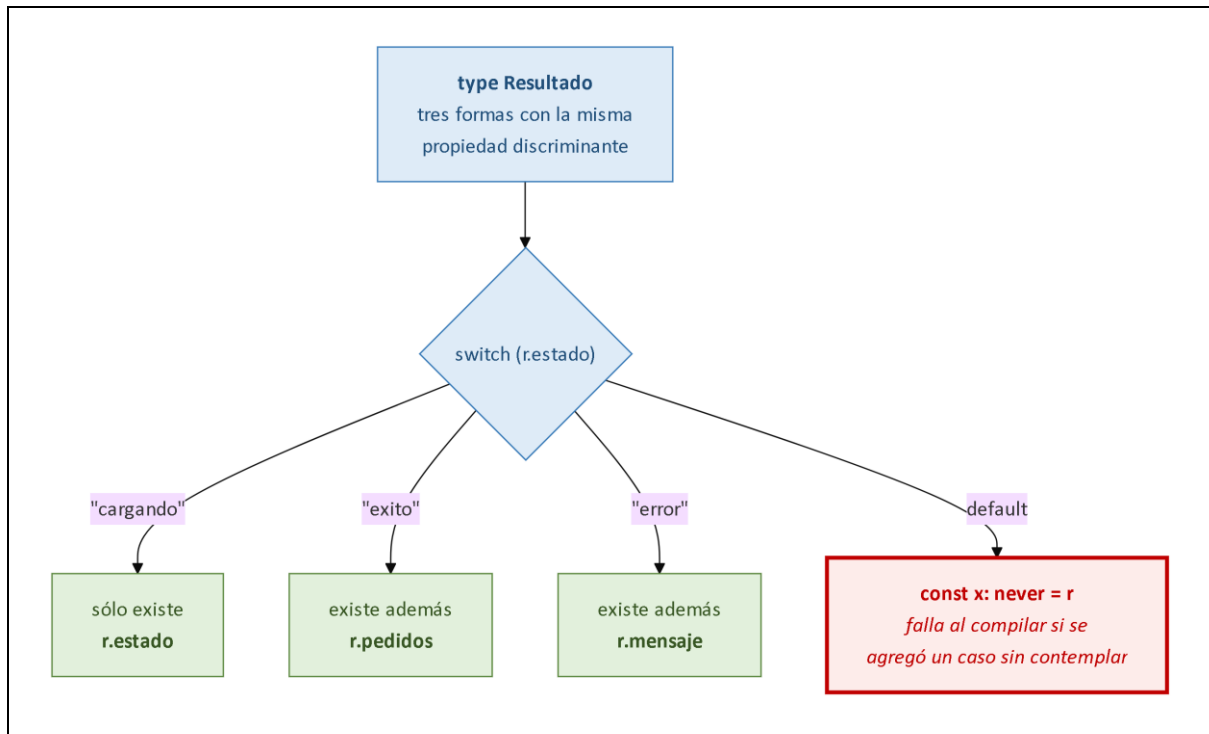


Figura 6.3. El estrechamiento de una unión discriminada

Y acá aparece el uso valioso de `never`, que convierte al compilador en guardián de la exhaustividad:

```
function render(r: Resultado) {
  switch (r.estado) {
    case "cargando": return mostrarCargando();
    case "exito":    return mostrarLista(r.pedidos);
    case "error":   return mostrarError(r.mensaje);
    default:
      const imposible: never = r; // error si se agregó un caso sin contemplar
      throw new Error(`Estado no contemplado: ${imposible}`);
  }
}
```

Si contemplaste los tres casos, al llegar al `default` no queda alternativa viva y el tipo de `r` es `never`. Si mañana se agrega un cuarto estado y alguien olvida sumarlo al `switch`, **el compilador falla en esa línea**. Sin ese `default`, el olvido pasa en silencio y produce una pantalla en blanco.

PARA ENTENDER: no te evita bugs, te evita olvidarte de lugares

El caso `default` **no maneja un error**: hace que el compilador te avise cuando agregaste un estado nuevo y te olvidaste de contemplarlo en algún lado.

Pensalo en el TPI. Si mañana la cátedra agrega un sexto estado —"demorado"— tenés que actualizar **todas** las vistas que lo muestran. Sin este patrón las buscás a mano y **te vas a olvidar de una**; con él **el compilador te las lista** y no te deja compilar hasta arreglarlas.

Eso es lo que TypeScript te da de verdad: **no te evita escribir bugs, te evita olvidarte de lugares**.

6.7 — Genéricos, con moderación

Un genérico permite escribir algo que funciona con muchos tipos **sin perder la información del tipo concreto**:

```
async function pedir<T>(ruta: string): Promise<T> {
  const respuesta = await fetch(ruta);
```

```

    if (!respuesta.ok) throw new ErrorDeApi(respuesta.status);
    return respuesta.json() as T;
}

const productos = await pedir<Producto[]>("/api/v1/productos");
// productos es Producto[], no any

```

Sin el genérico devolvería `any` y **cada uso perdería el tipo**: la cadena de contagio de la sección 6.4.2, justo donde entran todos los datos. La imagen es **la caja de mudanza**: se fabrica igual para todos, y lo que cambia es **la etiqueta**.

También se pueden **restringir** con `extends`, lo que permite usar propiedades adentro:

```

function porId<T extends { id: number }>(items: T[], id: number): T | undefined {
    return items.find(item => item.id === id);
}

```

Leelo así: «sirve para cualquier `T`, **siempre que tenga un `id` numérico**». Sin la restricción no podrías escribir `item.id`.

OJO ACÁ: los genéricos son adictivos

Son el lugar donde el código de TypeScript se vuelve ilegible, y casi siempre por la misma razón: **se agregan por si acaso**.

Si tenés que pensar treinta segundos para leer un tipo con tres parámetros genéricos anidados, la pregunta no es cómo simplificarlo: es **cuántos tipos distintos van a pasar realmente por ahí**. Si la respuesta es "uno", no necesitás un genérico: **necesitás ese tipo**.

Donde sí valen la pena, y en el TPI es un caso solo: **la capa `api/`**, donde una única función devuelve `Producto[]`, `Pedido` o `Usuario` según a quién se llame.

Como criterio: **la mayoría del código no los necesita**. Un genérico anidado en tres niveles indica que **la abstracción está mal puesta**, no que el problema sea complejo. Y fijate en el `as T` del primer ejemplo: **eso es una mentira al compilador**, y la sección 6.10 explica por qué ahí es inevitable.

6.8 — El modo estricto: qué prende cada bandera

El TPI declara en su sección 1.3 que el frontend usa **tipado estricto**: no es una recomendación de estilo, es **una opción del compilador** que activa un conjunto de verificaciones.

Opción	Qué exige	Qué error atrapa
<code>noImplicitAny</code>	Anotar todo parámetro sin tipo evidente	Un <code>any</code> que nadie pidió, y la cadena de 6.4.2
<code>strictNullChecks</code>	<code>null</code> y <code>undefined</code> no son asignables a cualquier tipo	El <i>Cannot read properties of null</i> del Capítulo 4
<code>strictFunctionTypes</code>	Verificación más estricta de parámetros	Pasar una función específica donde va una general
<code>strictPropertyInitialization</code>	Inicializar las propiedades de clase	Un campo que queda <code>undefined</code> en silencio
<code>noImplicitThis</code>	<code>this</code> no puede quedar implícito en <code>any</code>	Las rarezas de <code>this</code> del Capítulo 3
<code>useUnknownInCatchVariables</code>	La variable de un <code>catch</code> es <code>unknown</code>	Suponer que lo capturado es un <code>Error</code>

La decisiva es la segunda: sin ella `null` es assignable a todo y esto compila sin protestar:

```

function nombreDelCliente(pedido: Pedido): string {
    return pedido.cliente.nombre; // ¿y si cliente es null?
}

```

Con la verificación activada, si el tipo declara que `cliente` puede faltar esa línea **no compila**: te obliga a decidir **qué pasa cuando no hay cliente**, en lugar de que lo descubras con la pantalla en blanco. Existe por separado por una razón histórica —llegó en TypeScript 2.0, en 2016, cuando ya había mucho código que no la resistía—, pero hoy **no hay motivo para no activarla**, y el TPI la exige.

PARA ENTENDER: el detector de humo no hizo el humo

Cuando la actives en código que no la tenía, te van a aparecer decenas de errores de golpe y la reacción natural es pensar que la opción es demasiado molesta.

Dale vuelta la lectura: **cada uno de esos errores es un lugar donde tu programa puede explotar en producción**. No los creó la opción: **ya estaban**. Es un detector de humo que suena el día que lo instalás.

El error clásico es este, y lo viviste hace dos capítulos:

```
const boton = document.querySelector("#pagar");
boton.addEventListener("click", pagar); // ERROR: boton puede ser null
querySelector devuelve null cuando no encuentra nada, y el Cannot read properties of null aparece en ejecución. El compilador acaba de atajarte ese bug antes de que exista.
```

La última fila de la tabla también cambia un hábito:

```
try {
  await crearPedido(datos);
} catch (error) { // error es unknown, no any
  error.message; // error de compilación
  if (error instanceof ErrorDeApi) error.codigo; // así sí
}
```

Y es correcto: en JavaScript **se puede lanzar cualquier cosa**. Asumir que lo capturado tiene `message` es una suposición, y el modo estricto obliga a verificarla. Conecta con el Capítulo 5: ahí puede llegar un `TypeError` de red, un `AbortError` del plazo de espera o un error propio.

6.9 — Del contrato del TPI al tipo

6.9.1 — Traducir un esquema de la sección 7

La sección 7 del TPI define los esquemas de request y response de cada endpoint, y no es documentación complementaria: **es la especificación de los tipos del frontend**. Traducirla es la tarea central de esta clase.

```
// Estados: unión literal, no string (sección 3.4 del TPI)
export type EstadoPedido =
  | "pendiente" | "confirmado" | "en_preparacion" | "entregado" | "cancelado";

export interface PedidoResponse {
  id: number;
  estado: EstadoPedido;
  total: string; // ← cadena decimal. Ver 6.9.2
  creado_en: string; // fecha en formato ISO
  items: ItemPedidoResponse[];
  direccion: DireccionResponse | null;
}
```

Tres decisiones ya visibles. `estado` es una unión literal y no `string` (sección 6.6). Las fechas son `string` y no `Date`: **lo que llega por la red es texto**, y declararlo `Date` sería declarar algo falso. Y el `| null` declara que la dirección **puede no estar**: con el modo estricto, el compilador obliga a contemplarlo.

PARA ENTENDER: el tipo describe lo que hay, no lo que te gustaría

Lo de las fechas parece un detalle y **es el mismo principio que RN-F08** en otro campo. Si declarás `creado_en: Date`, el compilador te deja escribir `pedido.creado_en.getFullYear()`. Compila perfecto. **Y explota en ejecución**, porque lo que hay ahí es la cadena `"2026-08-25T14:30:00Z"`. **El tipo mintió porque vos se lo pediste.**

La regla vale para todo el módulo: **el tipo describe lo que realmente hay, no lo que te gustaría que hubiera.** ¿Querés un `Date`? Lo construís vos, en la capa `api/` — el mismo lugar donde se convierte el importe de RN-F08.

6.9.2 — El dinero, y por qué el tipo es el garante de RN-F08

Acá converge todo. La sección 3.5.4 estableció que el tipo numérico de JavaScript **no representa exactamente ciertos decimales** y que ese error minúsculo se acumula. Sobre esa base, la regla del TPI dice:

RN-F08. Todo importe recibido llega como string decimal y se convierte a number en la capa `api/`, nunca en la vista; ninguna operación aritmética sobre dinero ocurre en el frontend **salvo el subtotal de exhibición del carrito**. Garante: revisión, y **el tipo TypeScript, que declara string**.

Esa última frase le da sentido al capítulo entero. **El tipo no es documentación: es el mecanismo que hace cumplir la regla.**

```
interface PedidoResponse {
  total: string;      // ← el garante
}

const p: PedidoResponse = await pedir("/api/v1/pedidos/1043");
const doble = p.total * 2;  // ERROR de compilación
```

Si el tipo dijera `number`, esa multiplicación **compilaría sin una queja**, daría un resultado con error de punto flotante, y nadie se enteraría hasta que las cuentas del mes no cerraran. **Con string, el compilador la rechaza** — no porque entienda de dinero, sino porque **no se puede multiplicar texto: la mejor manera de impedir una operación prohibida es hacerla imposible de escribir.**

La regla nombra además una excepción: **el subtotal de exhibición del carrito**, enunciada con todas las letras porque **es una excepción, no un permiso general**. **El total que se cobra lo calcula el servidor.**

Este es el punto que el Capítulo 3 te anticipó

Cuando le pidas a un agente de IA que te tipe la respuesta del endpoint de pedidos, **te va a escribir total: number**. Sin dudar. Y va a tener razón en un sentido: **es lo que hace todo el mundo**, y `total` suena a número.

Lo que el agente no sabe es que **ese campo cruzó una frontera** donde el punto flotante rompe la exactitud, y que hay una regla que lo declara texto y nombra al tipo como su garante. **Vos sí. Ese es el módulo entero en un campo de un tipo.**

6.9.3 — Que un identificador no se pueda pasar por otro

Cuando hay muchas entidades —el TPI tiene veintitrés— aparece un problema: **todos los identificadores son number** y, por el tipado estructural, son intercambiables.

```
function cancelarPedido(pedidoId: number) { ... }
cancelarPedido(producto.id);  // compila. Y está mal.
```

Es la cerradura mostrando su costo. Un **tipo de marca** le agrega un diente extra: una propiedad imposible que **sólo existe para el sistema de tipos**.

```
type PedidoId = number & { readonly __marca: "PedidoId" };
type ProductoId = number & { readonly __marca: "ProductoId" };

function cancelarPedido(id: PedidoId) { ... }
cancelarPedido(producto.id); // ERROR: ProductoId no es PedidoId
```

¿Y en ejecución? **Nada**. Como los tipos se borran (sección 6.3), **siguen siendo números comunes**: la marca no existe fuera del compilador y no cuesta un byte. Es una técnica avanzada y **no hace falta aplicarla en todas partes**: vale cuando circulan muchos identificadores del mismo primitivo, que es el caso del TPI.

6.9.4 — Cómo queda la capa `api/`

```
export async function obtenerPedido(id: PedidoId): Promise<Pedido> {
  const respuesta = await fetch(`/api/v1/pedidos/${id}`, {
    headers: { Authorization: `Bearer ${token()}` },
    signal: AbortSignal.timeout(10_000),
  });
  if (!respuesta.ok) throw await construirError(respuesta);
  const crudo = (await respuesta.json()) as PedidoResponse;
  return aPedidoDeDominio(crudo); // acá, y sólo acá, se convierte
}
```

Leela despacio, porque **es el módulo entero en nueve líneas**: el token del Capítulo 1, porque el protocolo no se acuerda de nada; la verificación de `respuesta.ok` del Capítulo 5, porque `fetch` no rechaza ante un 404 ni un 500; el plazo de espera; el identificador marcado de la sección 6.9.3; y **la conversión de RN-F08 en un solo lugar**. **Ninguna vista repite nada de esto**: repartida en seis, alcanzaría con que una se olvidara.

6.10 — Dónde mienten los tipos

Es la sección más importante del capítulo y la que casi nunca se enseña: todo lo anterior te dio herramientas, esta te dice **dónde dejan de funcionar**.

6.10.1 — El borde de la red

```
const datos = await respuesta.json();
```

`json()` devuelve `any`, y no puede hacer otra cosa: parsea texto en tiempo de ejecución, y el compilador no tiene forma de saber qué va a venir. En cuanto ese valor se asigna a algo tipado:

```
const pedido = (await respuesta.json()) as PedidoResponse;
```

Esa línea no comprueba nada. Es una afirmación del programador: "confía en que esto tiene esta forma". Si el servidor renombró un campo, si un opcional vino nulo, si la respuesta trae otra estructura, **el compilador no se entera**, y el error aparece después, en cualquier parte, con un mensaje que no menciona el origen.

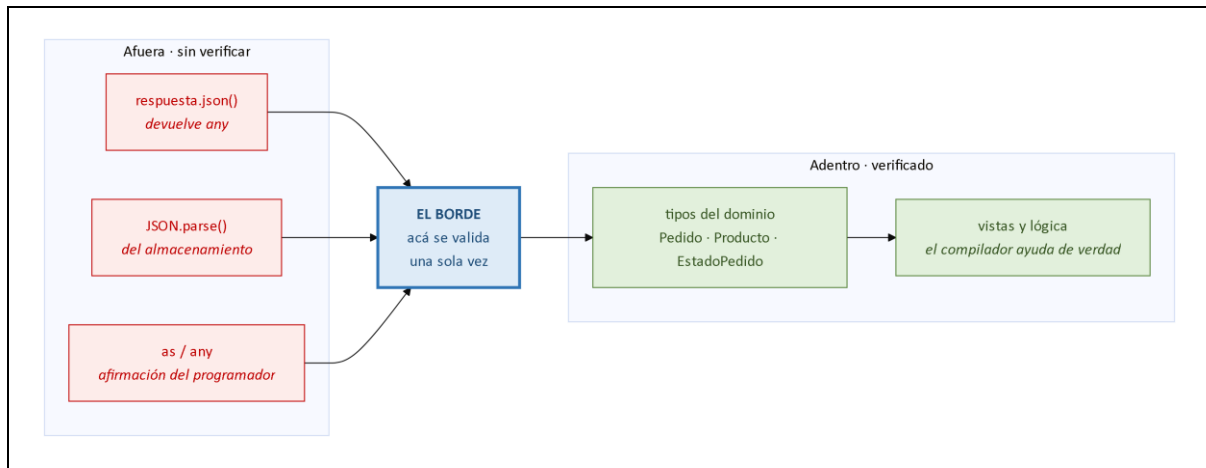


Figura 6.4. Dónde mienten los tipos

Entrada	Por qué miente
<code>respuesta.json()</code>	Devuelve <code>any</code> ; nadie comprueba la forma
<code>JSON.parse()</code>	Ídem: lo guardado en el navegador puede ser de otra versión
<code>as</code> y <code>any</code>	Son afirmaciones del programador, no verificaciones

El segundo **muerde en el arranque**: lo que se guardó con una versión anterior **sigue ahí** cuando el código cambió, se lee con la forma nueva y nada avisa. En tu máquina no pasa, porque limpiaste el almacenamiento hace dos semanas.

Verificación en tiempo de compilación	Verificación en tiempo de ejecución
Antes de que el programa exista	Mientras corre, sobre los datos reales
La hace el compilador, gratis	La escribís vos: <code>typeof</code> , <code>in</code> , campo por campo
Alcanza a lo que escribiste	Alcanza a lo que entra de afuera
Desaparece al compilar	Viaja al navegador como código común
Si falla, te lo dice el editor	Si falla, lo descubre el usuario

6.10.2 — Validar en el borde, una sola vez

La solución es simple de enunciar: **verificar en ejecución, una sola vez, en el punto de entrada**. Una función que recibe `unknown`, comprueba, y devuelve el tipo o falla:

```
function aPedidoResponse(dato: unknown): PedidoResponse {
  if (typeof dato !== "object" || dato === null) {
    throw new ErrorDeContrato("Se esperaba un objeto");
  }
  const d = dato as Record<string, unknown>;
  if (typeof d.id !== "number") throw new ErrorDeContrato("id");
  if (typeof d.total !== "string") throw new ErrorDeContrato("total");
  if (!ESTADOS.includes(d.estado as EstadoPedido)) {
    throw new ErrorDeContrato(`estado desconocido: ${d.estado}`);
  }
  return dato as PedidoResponse;
}
```

Fijate en el detalle que hace que funcione: **el parámetro es `unknown` y no `any`**. Con `any` la función sería decorativa; con `unknown` **te obliga** a verificar antes de tocar nada. Y si falla, el error está **en la línea donde el dato entró**, con el nombre del campo. No trescientas líneas después, en una vista, diciendo "undefined".

El TPI **no declara en su stack ninguna biblioteca para validar respuestas de la API**; sí declara `@tanstack/form-core`, que valida **formularios**, que es otro problema. Esta verificación es **código propio** y hay que dosificarla, en dos lugares:

- **Las respuestas de los endpoints críticos**, los que mueven dinero o cambian estados: un catálogo mal tipado muestra un producto raro, un pedido mal tipado cobra mal.
- **Lo que se lee del almacenamiento del navegador al arrancar**, donde el desajuste de versiones es más probable.

Y la validación **real** está en el servidor, con los esquemas de la sección 7 del TPI. Lo del cliente es **defensa contra el desajuste de contrato, no contra un atacante** — porque, como estableció el Capítulo 5 y antes la regla RN-F04, **el atacante no usa tu cliente**.

OJO ACÁ: cada `as` es una promesa que hacés vos

Esta es la trampa mental más peligrosa de TypeScript: **ves los tipos, ves que compila, y creés que el dato tiene esa forma**. No. Vos **declaraste** que la tiene.

```
const pedido = (await respuesta.json()) as PedidoResponse;
```

Ahí no hay ninguna verificación. **Cero**. Si el backend renombra `total` a `monto_total`, tu tipo sigue diciendo `total`, todo compila igual de lindo y en pantalla aparece `undefined`. El compilador no falló: **hizo lo que le pediste**.

Regla para todo el módulo: **cada `as` que escribas es una promesa tuya, no una verificación de la herramienta**. Contalos: cada uno es un lugar donde podés estar equivocado.

6.11 — Herramientas de diagnóstico

En el Capítulo 1 el instrumental estaba en el navegador; acá está en el editor.

Pasar el cursor sobre un identificador muestra el tipo inferido, más informativo que el declarado: dice lo que el compilador entendió, no lo que creíste declarar. Es la forma de descubrir que una variable quedó en `any` (sección 6.4.2).

FIGURA 6.6

captura

Captura del editor mostrando el tipo inferido de una variable. Idealmente una que quedó en `any` sin que nadie lo pidiera, que es lo que hay que aprender a detectar

Figura 6.6. El tipo inferido al pasar el cursor

Ir a la definición y buscar todas las referencias funcionan sobre el análisis de tipos y no sobre búsqueda de texto, y de ahí sale **la razón por la que renombrar deja de dar miedo** — uno de los tres problemas que motivaron TypeScript en 2010 (sección 6.2).

El **compilador en línea de comandos** conviene ejecutarlo en modo de sólo verificación, sin emitir archivos, y dejarlo en modo de observación durante el desarrollo. Es lo que evita la situación de la sección 6.3.

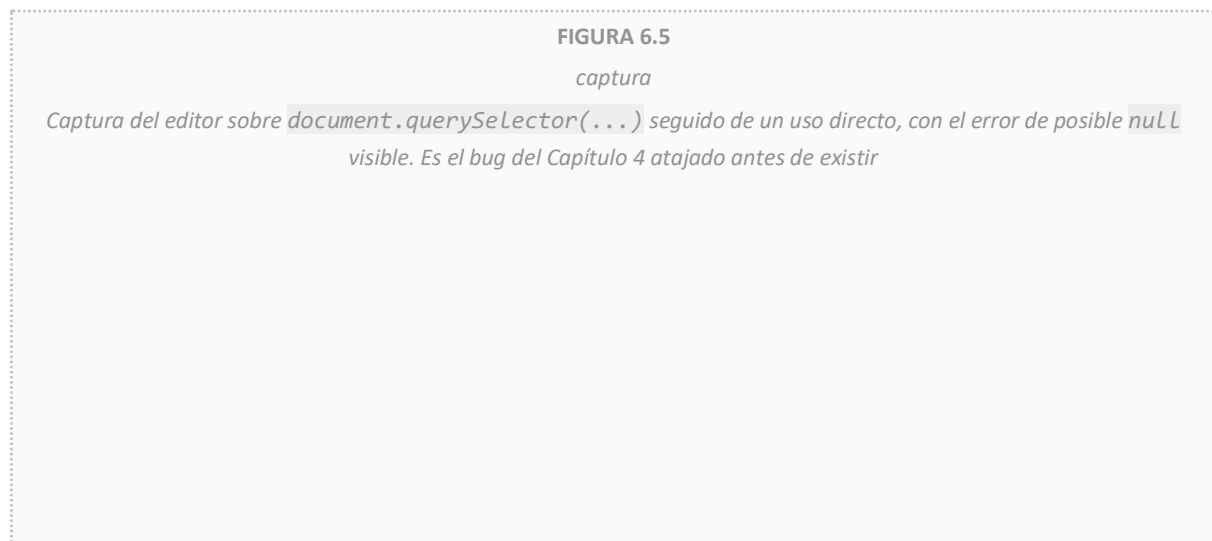


Figura 6.5. Un error de verificación de nulos

Dos opciones más atrapan **una clase de error que el modo estricto no cubre**. Una advierte sobre el **acceso a un índice de un arreglo**: `productos[47]` tiene tipo `Producto` aunque el arreglo tenga tres elementos. La otra, sobre **propiedades opcionales asignadas explícitamente como undefined**. Saber qué queda fuera del modo estricto es parte de saber hasta dónde te protege.

6.12 — Seguridad y evolución

La consideración	Por qué importa
Los tipos no son un control de seguridad	Se borran: no existen en ejecución . Un atacante no compila tu código, emite peticiones. La validación con consecuencias vive en el servidor: RN-F04 otra vez
<code>any</code> es una renuncia, y se propaga	Desactiva la verificación de todo lo derivado , sin señal. El modo estricto prohíbe los implícitos; los explícitos son legales
Las dependencias traen sus propios tipos, de calidad variable	Un paquete puede declarar tipos que no corresponden a su comportamiento real , y el compilador confía como en un <code>as</code>

Sobre la evolución, dos incorporaciones y una tendencia de fondo.

El operador `satisfies` verifica que un valor cumple un tipo **sin perder la inferencia del tipo concreto**: una tensión clásica entre anotar y dejar inferir.

Las **anotaciones de tipo como comentarios** son una propuesta ante el comité de ECMAScript —el mismo que abandonó ECMAScript 4— para que **el lenguaje ignore explícitamente las anotaciones** y el navegador ejecute código anotado sin compilar. Si prospera, cambiaría el paso de construcción del Capítulo 7.

Y la tendencia de fondo: cada vez más herramientas derivan los tipos **directamente de la especificación de la API**, de un documento OpenAPI por ejemplo. Eso ataca el problema de la sección 6.10: **si los tipos se generan desde el contrato, no pueden desincronizarse de él**.

6.13 — Verificación: el checklist honesto

Nueve comprobaciones. **No son ejercicios: son el criterio para saber si el capítulo se entendió.** Cada una cierra con la sección que la funda.

- Compilar un archivo con un error de tipo deliberado y **verificar que el JavaScript se emitió igual.** (6.3)
- Declarar lo mismo con `const` y con `let`, y explicar **por qué el tipo inferido difiere.** (6.4.1)
- Escribir una función con parámetro `unknown` y otra con `any`, y documentar **qué permite cada una.** (6.4.2)
- Traducir un esquema de la sección 7 del TPI a una interfaz, con **la unión literal para los estados y string para los importes.** (6.9.1)
- Intentar una operación aritmética sobre un importe y **verificar que el compilador la rechaza.** (6.9.2)
- Implementar una unión discriminada con exhaustividad, agregar un caso y **comprobar que el compilador señala el switch incompleto.** (6.6)
- Escribir un `as` sobre `json()`, provocar un desajuste y documentar **dónde aparece el error y cuánto se aleja de su causa.** (6.10.1)
- Activar la verificación estricta de nulos donde no estaba y documentar **cuántos errores aparecen y de qué tipo.** (6.8)
- Pasar el cursor sobre cinco variables y verificar que **ninguna quedó inferida como any.** (6.11)

6.14 — Los once errores frecuentes

Todos tienen algo en común: **en el momento no parecen errores.**

El error	Por qué duele	Sección
Creer que si compila, funciona	El sistema no es sólido por decisión , y los datos externos no se verifican	6.2 y 6.10
Usar <code>any</code> para que deje de subrayar	Se propaga a todo lo derivado. Para lo desconocido va <code>unknown</code>	6.4.2
Confiar en un <code>as</code> sobre <code>json()</code>	Es una afirmación, no una verificación: el error aparece lejos de su causa	6.10.1
Declarar los importes como <code>number</code>	Reintroduce el punto flotante y anula el garante que el TPI declara	6.9.2
Declarar las fechas como <code>Date</code>	Lo que llega es texto : compila y explota al llamar <code>getFullYear</code>	6.9.1
Usar <code>string</code> para los estados	Pierde el autocompletado y admite valores inexistentes	6.6
Omitir la verificación de exhaustividad	Al agregar un estado, los <code>switch</code> incompletos fallan en silencio	6.6
Anotar todo, incluso lo obvio	Ruido y una cosa más para desactualizar	6.4.1
Abusar de los genéricos	Un genérico anidado indica una abstracción mal puesta	6.7

El error	Por qué duele	Sección
Suponer que un <code>catch</code> captura un <code>Error</code>	Se puede lanzar cualquier cosa; el modo estricto lo tipa <code>unknown</code>	6.8
Confiar en la herramienta de construcción	No verifica los tipos: los borra	6.3

6.15 — Las actividades, y qué busca cada una

1. Del esquema al tipo

Tomar tres esquemas de la sección 7 del TPI —pedidos, catálogo, usuarios— y escribir sus interfaces completas **documentando cada decisión**: por qué una unión literal, por qué `string` en los importes, por qué opcional o no.

Qué busca: *que dejes de copiar el esquema y empieces a traducirlo: cada campo es una decisión, y la que no justificás es la que te muerde.*

2. El garante en acción

Escribir una función que calcule el total de un carrito a partir de las respuestas del servidor y documentar **en qué línea exacta** el compilador impide la aritmética. Implementar después la excepción de RN-F08 —el subtotal de exhibición— y **justificar dónde ocurre la conversión**.

Qué busca: *sentir al garante trabajando: una cosa es leerlo y otra que te lo impida a vos.*

3. Exhaustividad obligatoria

Implementar la vista de estado de un pedido con unión discriminada y exhaustividad. Agregar un sexto estado y documentar **todos** los lugares que el compilador señala.

Qué busca: *ver la lista de lugares olvidados apareciendo sola.*

4. Capa `api/` tipada

Escribir una función genérica de acceso a la API con el token, la verificación de `respuesta.ok` del Capítulo 5, el plazo de espera y el tipado del resultado. Usarla en cuatro endpoints y **verificar que ninguna vista repite esa lógica**.

Qué busca: *ver que la capa no es burocracia: es donde el garante de RN-F08 actúa una sola vez para todo el sistema.*

5. Identificadores distinguibles

Implementar tipos de marca para tres identificadores del TPI y demostrar que el compilador rechaza pasar uno donde va otro. **Verificar en el JavaScript emitido que la marca desapareció**.

Qué busca: *las dos mitades juntas: el tipo protege mientras escribís y no existe cuando el programa corre.*

6. Exploración: dónde mienten los tipos

Tomar un endpoint del TPI, tiparlo y **renombrar deliberadamente un campo en la respuesta del servidor**. Documentar en qué punto aparece el error, cuánto se aleja de su causa y qué mensaje da. Escribir después una validación en el borde y repetir. *(Requiere el backend del TPI o un servidor simulado.)*

Qué busca: *buscar un error en el lugar equivocado y después encontrarlo en el primer intento: la sección 6.10 vivida en lugar de leída.*

7. Exploración: la solidez que no está

Buscar en los objetivos de diseño de TypeScript **la lista de objetivos no perseguidos** e identificar tres construcciones que puedan compilar y fallar en ejecución. Para cada una, un ejemplo mínimo y **qué se ganó permitiéndola**. (*Requiere el repositorio del proyecto.*)

Qué busca: *cada agujero está ahí a cambio de algo: es la pregunta del Capítulo 1, ¿qué resignó para funcionar?*

6.16 — Síntesis: las once frases

1. TypeScript no resuelve que JavaScript permita sumar un número y un texto: resuelve que **a escala nadie recuerda qué forma tiene cada objeto**. Es un problema de memoria humana, no de aritmética.
2. Dart fracasó porque exigía reescribir todo. **TypeScript es un superconjunto**, y esa decisión — adopción sin migración — explica su éxito.
3. El tipado es **estructural**: importa la forma, no el nombre. Es la cerradura y no el guardia con la lista, coherente con el modelo de objetos del Capítulo 3.
4. **Los tipos se borran**. Son las marcas de lápiz del carpintero: no cuestan nada y no garantizan nada en ejecución.
5. **El sistema de tipos no es sólido, por decisión explícita**, a cambio de poder tipar JavaScript real. Es una herramienta de productividad, no una prueba.
6. `any` desactiva la verificación y **se propaga**; `unknown` expresa la misma incertidumbre sin renunciar a nada. Para lo que entra de afuera, `unknown`.
7. La **unión discriminada con verificación de exhaustividad** convierte al compilador en un buscador de lugares olvidados: lo más valioso del capítulo para un sistema con una máquina de estados.
8. **El tipo `string` en los importes es el garante de RN-F08**, y el TPI lo dice con esas palabras. Un `number` ahí reintroduce el error de punto flotante del Capítulo 3 y nadie se entera hasta que las cuentas no cierran.
9. **Los tipos mienten en el borde**. `json()` devuelve `any` y un `as` es una afirmación del programador: cada uno es un lugar donde se puede estar equivocado.
10. Validar en ejecución, **una sola vez y en el punto de entrada**, cierra ese hueco. La validación con consecuencias sigue estando en el servidor.
11. Un agente de IA va a escribir `total`: `number` con total seguridad, porque es lo más común. **Lo más común y lo correcto no siempre coinciden**, y distinguirlos es el objetivo de todo el módulo.

6.17 — Qué leer, y en qué orden

TypeScript no tiene una especificación normativa vigente: la que existió se abandonó en 2016 por imposible de mantener al ritmo del lenguaje, de modo que **la implementación es la referencia**. Por eso acá no hay una RFC que citar como en el Capítulo 1.

Si lees una sola cosa

Los objetivos de diseño de TypeScript, publicados en la wiki del repositorio como *TypeScript Design Goals*. Son los que se citan en la sección 6.2, **incluida la declaración de no perseguir un sistema de tipos demostrablemente correcto**. Diez minutos que cambian cómo vas a usar la herramienta.

Si lees tres

- **La documentación oficial** en typescriptlang.org/docs, que cumple el papel de la especificación que no existe: su *Handbook* cubre la inferencia, el estrechamiento y los genéricos de las secciones 6.4 a 6.7.
- **Cherny, *Programming TypeScript*** (O'Reilly, 2019): la mejor introducción sistemática **para quien ya programa**; su capítulo sobre el sistema de tipos explica el tipado estructural con más rigor que la documentación.
- **Vanderkam, *Effective TypeScript*** (2.ª edición, O'Reilly, 2024): en recomendaciones breves, trata los temas de la sección 6.10 — **el borde de los datos externos, el costo de `any` y cuándo una aserción es legítima**.

Las fuentes normativas (para consultar, no para leer de corrido)

- **La referencia de `tsconfig`**: documenta una por una las opciones de la sección 6.8, **con ejemplos de qué error atrapa cada una**.
- **ECMA-262**: el lenguaje que TypeScript extiende sigue siendo ese, y **todo lo que este capítulo no menciona sobre semántica de ejecución está allí**. Es la norma del Capítulo 3.
- **Type Challenges**, en su repositorio público: ejercicios graduados que **exceden lo que este módulo necesita**, pero muestran hasta dónde llega el sistema de tipos.
- **La documentación de alguna biblioteca de validación en ejecución** —aunque el TPI no declare ninguna— para conocer el patrón de **derivar el tipo estático a partir de un esquema verificable**, la respuesta madura al problema de la sección 6.10.

Cierre: las seis cosas que hay que recordar

Si dentro de un mes te acordás de seis frases, que sean estas.

LAS SEIS

1. **TypeScript no te promete que si compila, anda**. Lo declararon ellos: es un corrector ortográfico muy bueno, no una prueba matemática.
2. **Los tipos se borran**. Son las marcas de lápiz del carpintero: sirven mientras construís y no existen en el mueble terminado.
3. **`any` es un agujero que se agranda solo**. Para lo que no sabés qué es, `unknown`, que te obliga a averiguarlo antes de usarlo.
4. **La unión discriminada con `never` no te evita bugs: te evita olvidarte de lugares**.
5. **`total: string` es el garante de RN-F08**. No se puede multiplicar texto, y esa restricción tonta es la que hace falta.
6. **Cada `as` es una promesa tuya, no una verificación de la herramienta**. Contalos.

Y una séptima, que no está escrita pero está en todas sus páginas: **los tipos describen lo que hay, no lo que te gustaría que hubiera**. El día que declares un `Date` donde llega una cadena, el compilador te va a acompañar en la mentira hasta el final.

Continúa en: Capítulo 7 — Herramientas y componentes: Vite, Web Components y Chart.js, donde el código tipado de este capítulo se convierte en algo que el navegador puede ejecutar, y donde las suscripciones del Capítulo 4 encuentran el lugar donde darse de baja.